Lecture Notes: Dimensionality Reduction and Visualization Part II

Course: 2110573 Pattern Recognition

2026-02-15
version 0.9

1 Introduction to Dimensionality Reduction

The primary goal of dimensionality reduction is to transform data from a high-dimensional space into a lower-dimensional space while preserving some meaningful properties of the original data. This is crucial for visualization, computational efficiency, and mitigating the "curse of dimensionality."

2 Recap: Principal Component Analysis (PCA)

PCA is a fundamental dimensionality reduction technique that aims to find a projection (a linear transformation) that captures the maximum amount of variation in the data.

2.1 Core Idea

- **Objective:** Reduce dimensionality while retaining as much useful information as possible.
- **What is useful information?** In the context of PCA, it's the variance or "scatter" of the data.
- **Method:** Find a set of orthogonal axes (principal components) that align with the directions of maximum variance in the data. The first principal component is the direction of the largest variance.

2.2 Mathematical Formulation

We want to find a projection vector $\mathbf{w}$ that maximizes the variance of the projected data.

$$\underset{\mathbf{w}}{\operatorname{argmax}} \operatorname{Var}(\mathbf{w}^T\mathbf{x}) \tag{1}$$

This optimization is subject to the constraint that $\mathbf{w}$ is a unit vector, i.e., $\|\mathbf{w}\| = 1$.

The solution to this problem is that $\mathbf{w}$ must be an eigenvector of the covariance matrix of the data, Σ.

$$\Sigma\mathbf{w} = \lambda\mathbf{w} \tag{2}$$

where λ is the corresponding eigenvalue.

2.3 Selecting Eigenvectors

The variance of the data projected onto an eigenvector $\mathbf{w}$ is given by:

$$\operatorname{Var}(\mathbf{w}^T\mathbf{x}) = \mathbf{w}^T\Sigma\mathbf{w} \tag{3}$$

By substituting $\Sigma\mathbf{w} = \lambda\mathbf{w}$, we get:

$$\mathbf{w}^T(\lambda\mathbf{w}) = \lambda\mathbf{w}^T\mathbf{w} = \lambda\|\mathbf{w}\|^2 = \lambda \tag{4}$$

This shows that the eigenvalue λ is equal to the variance captured by its corresponding eigenvector $\mathbf{w}$. Therefore, to capture the most variance, we should select the eigenvectors with the highest eigenvalues.

2.4 Mean Centering and Reconstruction

PCA models variance, which is variation around the mean. It is essential to first mean-center the data by subtracting the mean vector $\mathbf{m}$ from each data point $\mathbf{x}$.

- **Projection:** $\mathbf{p} = V^T(\mathbf{x} - \mathbf{m})$, where V is the matrix of selected eigenvectors.
- **Reconstruction:** To reconstruct the original data point (approximately), we project it back and add the mean: $\tilde{\mathbf{x}} = V\mathbf{p} + \mathbf{m}$.

2.5 Practical Issues: The Gram Matrix Trick

For high-dimensional data like images (e.g., $d \approx 400,000$), the covariance matrix Σ is enormous ($d \times d$). However, the rank of Σ is at most $N - 1$, where N is the number of training samples. If $N \ll d$, we can solve the eigenvalue problem more efficiently.

Let the mean-centered data matrix be X. The covariance matrix is $\Sigma = XX^T$. Instead of solving the eigenvalue problem for this $d \times d$ matrix, we can solve it for the $N \times N$ Gram matrix, X^TX.

1. Solve the eigenvalue problem for the Gram matrix: $X^TX\mathbf{v}' = \lambda\mathbf{v}'$.
2. Pre-multiply by X: $XX^T(X\mathbf{v}') = \lambda(X\mathbf{v}')$.
3. This shows that if $\mathbf{v}'$ is an eigenvector of X^TX, then $\mathbf{v} = X\mathbf{v}'$ is an eigenvector of $\Sigma = XX^T$ with the same eigenvalue λ.

This reduces the problem from a $d \times d$ scale to an $N \times N$ scale, which is much more manageable when $N \ll d$.

2.6 PCA for Classification

PCA is an **unsupervised** method; it does not use class labels. It finds directions of maximum variance, which may not be the directions that best separate classes. In some cases, the principal component can project different classes on top of each other, making them inseparable.

3 Linear Discriminant Analysis (LDA)

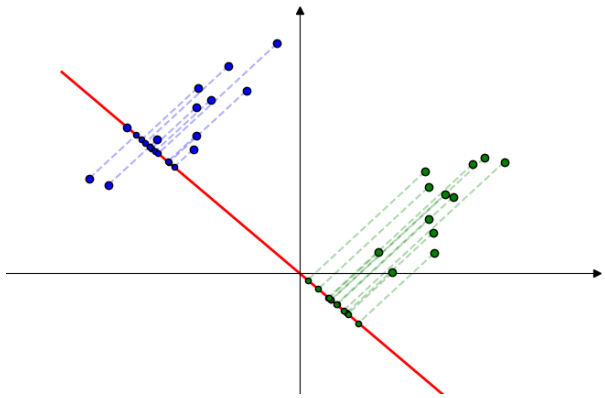

Figure 1: An example of LDA projection.

LDA is a **supervised** dimensionality reduction technique that explicitly tries to find a projection that best separates different classes.

3.1 Core Idea

- **Objective:** Find a projection that maximizes the distance between the means of the classes while minimizing the variance (scatter) within each class.
- **Assumption:** The data in each class follows a unimodal Gaussian distribution.

3.2 Mathematical Formulation (2-Class Case)

We want to maximize the Fisher Linear Discriminant criterion, $J(\mathbf{w})$:

$$J(\mathbf{w}) = \frac{\text{between-class scatter}}{\text{within-class scatter}} = \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}} \tag{5}$$

where:

- $S_B = (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)^T$ is the **between-class scatter matrix**.
- $S_W = \sum_{i \in C_1} (\mathbf{x}_i - \boldsymbol{\mu}_1)(\mathbf{x}_i - \boldsymbol{\mu}_1)^T + \sum_{j \in C_2} (\mathbf{x}_j - \boldsymbol{\mu}_2)(\mathbf{x}_j - \boldsymbol{\mu}_2)^T$ is the **within-class scatter matrix**.

The solution that maximizes $J(\mathbf{w})$ is found by solving the generalized eigenvalue problem:

$$S_B \mathbf{w} = \lambda S_W \mathbf{w} \quad \Rightarrow \quad S_W^{-1} S_B \mathbf{w} = \lambda \mathbf{w} \tag{6}$$

For a 2-class problem, this simplifies to: $\mathbf{w} \propto S_W^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)$.

3.3 LDA+PCA (Fisher Projection)

The within-class scatter matrix S_W can be singular if the number of samples N is less than the dimension d. A common approach to handle this is:

1. Use PCA to reduce the dimensionality to $N - C$ (where C is the number of classes), which makes S_W invertible.
2. Perform LDA in this lower-dimensional subspace to find the C-1 discriminant directions.

This combination is often called a Fisher projection.

4 Random Projection (RP)

Random Projection is a simple and powerful method that projects data onto a lower-dimensional subspace using a random matrix.

- **Method:** Project the original d-dimensional data to a k-dimensional subspace using a random $k \times d$ matrix R.
- **Johnson-Lindenstrauss (JL) Lemma:** This lemma states that if points in a vector space are projected onto a randomly selected subspace of suitably high dimension, the distances between the points are approximately preserved.
- **Properties:** RP is computationally much faster than PCA, especially for very high-dimensional data, and is robust to outliers. It is data-independent.

5 Visualization Techniques

While PCA, LDA, and RP are useful for downstream ML tasks, they may not be optimal for human interpretation and visualization. Methods like t-SNE and UMAP are specifically designed to create meaningful 2D or 3D representations of high-dimensional data.

5.1 t-Distributed Stochastic Neighbor Embedding (t-SNE)

t-SNE [2] is a non-linear technique for visualization that excels at revealing the local structure of data, such as clusters.

5.1.1 Core Idea

t-SNE models the similarity between high-dimensional data points as a probability distribution and then minimizes the divergence between this distribution and a similar distribution for the corresponding low-dimensional points.

5.1.2 Algorithm Steps

1. **High-Dimensional Similarities:** For each pair of points x_i, x_j, define a conditional probability $p_{j|i}$ that x_i would pick x_j as its neighbor based on a Gaussian distribution centered at x_i:

$$p_{j|i} = \frac{\exp(-||x_i - x_j||^2/2\sigma_i^2)}{\sum_{k \neq i} \exp(-||x_i - x_k||^2/2\sigma_i^2)} \tag{7}$$

   To handle outliers and ensure symmetry, a symmetric joint probability is used:

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N} \tag{8}$$

2. **Perplexity:** The variance σ_i^2 of the Gaussian is determined by a user-defined parameter called "perplexity". It can be thought of as a smooth measure of the effective number of neighbors for each point. It is defined using Shannon entropy $H(P_i)$:

$$Perp(P_i) = 2^{H(P_i)} \quad \text{where} \quad H(P_i) = -\sum_j p_{j|i} \log_2 p_{j|i} \tag{9}$$

   t-SNE performs a binary search to find the σ_i that produces the desired perplexity.

3. **Low-Dimensional Similarities:** In the low-dimensional space, define a similar joint probability q_{ij}. Instead of a Gaussian, t-SNE uses a Student's t-distribution with one degree of freedom. This heavier-tailed distribution helps alleviate the "crowding problem" (where points in high-D are equidistant, but low-D space cannot accommodate them):

$$q_{ij} = \frac{(1 + ||y_i - y_j||^2)^{-1}}{\sum_{k \neq l} (1 + ||y_k - y_l||^2)^{-1}} \tag{10}$$

4. **Optimization:** The algorithm minimizes the Kullback-Leibler (KL) divergence between the joint probability distributions P (high-D) and Q (low-D) using gradient descent:

$$J(y) = D_{KL}(P||Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}} \tag{11}$$

5.1.3 t-SNE Summary

- **Goal:** Preserves local neighbor structure.
- **Parameters:** Number of iterations, perplexity (typically 5-50).
- **Key Limitation:** t-SNE **does not learn a projection function** (unlike PCA or LDA). If you have a new data point, you must re-run the entire algorithm. It is primarily for visualization.

5.2 Uniform Manifold Approximation and Projection (UMAP)

UMAP [1] is a newer non-linear dimensionality reduction technique that is often faster than t-SNE and can preserve more of the global structure of the data.

5.2.1 Core Idea

UMAP is based on manifold learning theory. It constructs a high-dimensional graph representation of the data and then optimizes a low-dimensional graph to be as structurally similar as possible.

5.2.2 Algorithm Steps

1. **High-Dimensional Graph:** UMAP constructs a weighted graph where edge weights represent the likelihood that two points are connected. This assumes the data is uniformly distributed on a locally connected manifold. The distance metric is locally adaptive using the distance to the nearest neighbor ρ_i and a scaling factor σ_i:

$$w_i(X_i, X_j) = \exp\left(\frac{-(d(X_i, X_j) - \rho_i)}{\sigma_i}\right) \tag{12}$$

   Here, ρ_i ensures the weight to the nearest neighbor is 1 (connectivity), while σ_i controls the neighborhood size.

2. **Symmetrization:** The weights are made symmetric using a probabilistic union formula, interpreted as the probability that at least one point considers the other a neighbor:

$$w(X_i, X_j) = w_i(X_i, X_j) + w_j(X_j, X_i) - w_i(X_i, X_j)w_j(X_j, X_i) \tag{13}$$

3. **Low-Dimensional Representation:** UMAP initializes points in a low-dimensional space.

4. **Optimization:** It minimizes the cross-entropy between the high-dimensional weights (w_h) and low-dimensional weights (w_l) using stochastic gradient descent. This forces the low-dimensional representation to have a similar topological structure to the high-dimensional data:

$$C = \sum_{e \in E} \left[w_h(e) \log \left(\frac{w_h(e)}{w_l(e)} \right) + (1 - w_h(e)) \log \left(\frac{1 - w_h(e)}{1 - w_l(e)} \right) \right] \tag{14}$$

5.2.3 UMAP vs. t-SNE

- UMAP is generally faster than t-SNE.
- UMAP often does a better job of preserving the global structure of the data (e.g., preserving continuous transitions between clusters).
- Visualizations of datasets like MNIST often show that UMAP produces tighter, more separated clusters while also hinting at the relationships between those clusters.
- UMAP, like t-SNE, is primarily used for visualization, but it can maintain better global coherence.

6 Summary of Methods

- **PCA:** Unsupervised, linear. Finds directions of maximal variance. Good for feature reduction when variance is a good proxy for information.
- **LDA:** Supervised, linear. Finds directions of maximal class separability. Good for classification-focused feature reduction.
- **Random Projection:** Unsupervised, linear. Preserves distances between points. Very fast and scalable.
- **t-SNE:** Unsupervised, non-linear. Excellent at revealing local cluster structure. Computationally intensive and for visualization only.
- **UMAP:** Unsupervised, non-linear. A modern alternative to t-SNE that is faster and often better at preserving global structure.

7 Fundamental Concepts in Model Training

Before diving into model evaluation, it's crucial to distinguish between two types of variables we encounter during the machine learning model lifecycle.

7.1 Parameters vs. Hyper-parameters

- **Parameters**: These are values that are *learned by the algorithm* from the training data. They are the internal configuration variables of the model. For example:
  - The weights in a linear regression or neural network.
  - The means and variances of Gaussian distributions in a GMM.
  - The probabilities associated with bins in a histogram-based model.
- **Hyper-parameters**: These are configuration variables that are *set by the user before the training process begins*. They are external to the model and cannot be learned directly from the data. The choice of hyper-parameters governs the training process itself. For example:
  - The learning rate in gradient descent.
  - The number of neighbors in a k-NN classifier.
  - The number of trees in a random forest.
  - The choice between a parametric or non-parametric model.

  Hyper-parameters are typically tuned by evaluating the model's performance on a **validation set**.

8 Data Splitting Strategies

To properly evaluate a machine learning model's ability to generalize to new, unseen data, we must split our dataset into multiple subsets. Evaluating a model on the same data it was trained on is a methodological mistake, as it doesn't measure generalization performance.

8.1 Simple Train-Test Split

The most basic approach is to split the data into two parts:

- **Training Set**: Used to train the model (i.e., learn the parameters).
- **Test Set**: Used to evaluate the final performance of the trained model.

A common split ratio is 80% for training and 20% for testing. This ratio can be varied depending on the total size of the dataset.

- **Stratified Splitting**: A crucial enhancement to the simple split. It ensures that the distribution of the target classes is the same in both the training and test sets as it is in the original dataset. This is especially important for imbalanced datasets.

8.2 Cross-Validation (CV)

A single train-test split can be sensitive to how the data is partitioned. To get a more robust estimate of the model's performance, we can use cross-validation.

- In **k-fold Cross-Validation**, the dataset is randomly partitioned into k equal-sized subsets (or "folds").
- The model is then trained and evaluated k times. In each iteration, one fold is held out as the test set, and the remaining $k-1$ folds are used for training.
- The final performance metric is the average of the metrics computed in each of the k iterations.
- This reduces the variance of the performance estimate. Common values for k are 3, 5, and 10.

8.3 Leave-One-Out (LOO) CV

This is an extreme form of k-fold CV where k is equal to the number of samples in the dataset (N).

- In each of the N iterations, one sample is used for testing, and the remaining $N - 1$ samples are used for training.
- This provides a nearly unbiased estimate of performance but can be computationally very expensive.

8.4 The Three-Way Split: Train-Validation-Test

When we need to tune hyper-parameters, a simple train-test split is not enough. Using the test set to tune hyper-parameters would cause the model to overfit to the test set, and the final performance estimate would be overly optimistic.

- **Training Set**: Used to train the model with a given set of hyper-parameters.
- **Validation Set** (or Development Set): Used to evaluate the performance of models with different hyper-parameters and select the best ones.
- **Test Set**: Used only *once* at the very end to report the final, unbiased performance of the model that was selected using the validation set. **The test set must never be used for training or tuning.**

8.5 Nested Cross-Validation

For a highly robust evaluation of a complete modeling pipeline (including hyper-parameter tuning), we can use nested CV. This is particularly useful for smaller datasets where a single, fixed test set may not be reliable.

- **Outer Loop**: Splits the data into training and testing folds. This loop's purpose is to evaluate the performance of the overall pipeline.
- **Inner Loop**: For each outer training fold, it performs another round of cross-validation to find the best hyper-parameters. The model is trained on the full outer training fold with these best hyper-parameters.
- The model is then evaluated on the outer test fold. The process is repeated for all outer folds, and the scores are averaged.

8.6 Practical Guidance on Data Size

- **Test Data**: Should be large enough to provide a statistically significant performance estimate. It must be representative of the real-world use cases the model will encounter.
- **Validation Data**: Should be large enough to differentiate between models with slightly different performance. For instance, if a 0.1% difference in performance matters, you might need at least 1000 samples where the models differ.
- **Training Data**: Should be large enough to allow the model to learn the underlying patterns and achieve the target performance. The required size is hard to estimate in advance and is often determined empirically.

For more information on determining sample sizes, the following resources may be helpful:

- Keras Examples: Sample Size Estimation
- ACL 2023 Findings: Paper on Sample Size
- ICLR Paper on Learning Curves

9 Receiver Operating Characteristic (ROC) Analysis

In binary classification, many models output a continuous value (a score or probability) rather than a hard class label. We then use a threshold to convert this score into a class prediction. ROC analysis is a powerful tool for evaluating model performance across all possible thresholds.

9.1 Prediction Thresholds and the Confusion Matrix

Let's consider a binary classification task. After applying a threshold to the model's scores, each prediction falls into one of four categories:

- **True Positive (TP)**: The model correctly predicts the positive class.
- **False Positive (FP)**: The model incorrectly predicts the positive class (a "false alarm").
- **True Negative (TN)**: The model correctly predicts the negative class.
- **False Negative (FN)**: The model incorrectly predicts the negative class (a "miss").

From these, we define key metrics:

- **True Positive Rate (TPR)**, also known as **Recall** or **Sensitivity**: The fraction of actual positives that are correctly identified.

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

- **False Positive Rate (FPR)**, also known as **Fall-out** or **False Alarm Rate (FAR)**: The fraction of actual negatives that are incorrectly identified as positive.

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

- **Precision**: The fraction of positive predictions that are actually correct.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

9.2 The ROC Curve

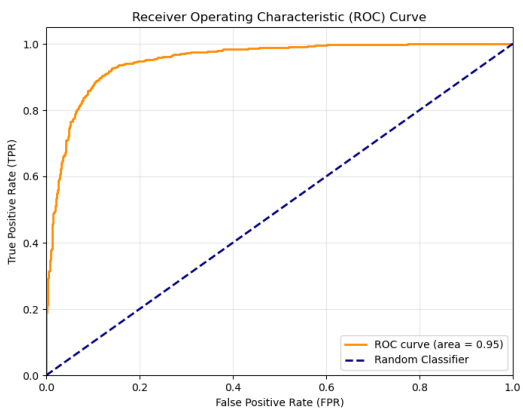

Figure 2: An example of ROC curve.

The ROC curve visualizes the trade-off between the benefits (TPR) and costs (FPR) of a classifier.

- It is a 2D plot with the **FPR on the x-axis** and the **TPR on the y-axis**.
- Each point on the curve corresponds to the (FPR, TPR) pair for a specific threshold. By varying the threshold from high to low, we trace the curve.

- A random classifier is represented by the diagonal line $y = x$.
- An ideal classifier would have a point at (0, 1), representing 100% TPR and 0% FPR.
- In general, models whose ROC curves are closer to the top-left corner are better.

9.3 Using ROC Curves for Model Comparison and Threshold Selection

- **Model Comparison**: If one model's ROC curve is entirely above another's, it is unambiguously better. If the curves cross, one model is better for certain ranges of FPR, while the other is better for others. The choice then depends on the specific application's requirements.
- **Threshold Selection**: The choice of an optimal threshold is application-dependent.
  1. **Hard Threshold**: Some applications have strict constraints, e.g., "the false alarm rate cannot exceed 1%." In this case, we find the point on the ROC curve where FPR = 0.01 and select the corresponding threshold to maximize TPR under that constraint.
  2. **Equal Error Rate (EER)**: If a miss (FN) is considered as bad as a false alarm (FP), we might choose the point where the False Negative Rate (FNR = 1 - TPR) equals the False Positive Rate (FPR). This is the point where the ROC curve intersects the line $y = 1 - x$.
  3. **Cost-Benefit Analysis**: In many real-world scenarios (e.g., medical diagnosis, churn prediction), the costs of false positives and false negatives are different. We can define a cost function and select the threshold that minimizes the expected cost. For example, if a false negative is 100 times more costly than a false positive, we would choose a threshold that makes 100(1-TPR) = FPR, resulting in a threshold that has a very low MR (high TPR).

9.4 Summarizing the ROC Curve

While the full curve is most informative, sometimes a single number is needed for comparison.

- **Area Under the Curve (AUC or AUROC)**: This is the most common metric. It represents the probability that the classifier will rank a randomly chosen positive instance higher than a randomly chosen negative one. AUC ranges from 0.5 (random) to 1.0 (perfect).
- **F-Score**: The harmonic mean of Precision and Recall. It's useful for finding a balance between these two metrics.

9.5 Related Curves

- **Precision-Recall (PR) Curve**: Plots Precision vs. Recall. It is particularly useful for tasks with a large class imbalance, where the ROC curve might be overly optimistic.
- **Detection Error Tradeoff (DET) Curve**: Plots FNR vs. FPR, usually on a normal deviate scale.

10 The Linear Classification Problem

In machine learning, a common task is classification, where we want to assign a label to an input data point. For a binary classification problem, we have data points belonging to one of two classes. A linear classifier attempts to solve this by finding a straight line (in 2D), a plane (in 3D), or a hyperplane (in higher dimensions) that separates the two classes.

As shown in Figure 3, for a given dataset that is linearly separable, there can be many possible hyperplanes that correctly classify the data. This raises a crucial question: **Which of these solutions is the best one?**

Figure 3: Multiple possible linear classifiers separating two classes. SVM aims to find the optimal one.

Intuitively, the best hyperplane is the one that is "furthest" from the data points of both classes. This provides a buffer zone, or a "margin," which leads to better generalization on unseen data. Logistic Regression, for example, finds a separating line but doesn't explicitly try to maximize this margin. It minimizes the squared error distance from all points to the line, which is a different objective.

11 Support Vector Machines (SVM)

Support Vector Machines (SVMs) formalize this intuition. The primary goal of an SVM is to **improve generalization** by finding the decision boundary that provides the most "slack" or margin between the two classes.

- **Goal:** Find the hyperplane that maximizes the margin.
- **Focus:** Instead of worrying about all data points, SVMs focus on the "borderline" cases—the points that are closest to the decision boundary. These critical points are called **Support Vectors**.

11.1 Defining the Margin

Let's define our linear classifier's decision boundary with the equation:

$$\mathbf{w}^T\mathbf{x} + b = 0$$

where $\mathbf{w}$ is a vector normal (perpendicular) to the hyperplane, and b is a bias term.

We can scale $\mathbf{w}$ and b freely without changing the hyperplane itself. SVM uses this property to define the margin boundaries. We select a scaling such that the decision boundary for the positive class is $\mathbf{w}^T\mathbf{x} + b = 1$ and for the negative class is $\mathbf{w}^T\mathbf{x} + b = -1$.

- For any positive sample ($y_i = +1$): $\mathbf{w}^T\mathbf{x}_i + b \geq +1$
- For any negative sample ($y_i = -1$): $\mathbf{w}^T\mathbf{x}_i + b \leq -1$

These two conditions can be combined into a single constraint:

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$$

The points for which this equality holds (i.e., they lie exactly on the margin boundaries) are the **support vectors**. The region between these two boundaries is the "margin," and no data points should fall within it in the linearly separable case.

11.2 Maximizing the Geometric Margin

The distance between the two margin boundaries, $\mathbf{w}^T\mathbf{x} + b = 1$ and $\mathbf{w}^T\mathbf{x} + b = -1$, can be calculated. This distance is the **geometric margin**. The margin width is given by:

$$\text{Margin} = \frac{2}{\|\mathbf{w}\|}$$

To maximize the margin, we must **minimize** $\|\mathbf{w}\|$. For mathematical convenience, we minimize $\|\mathbf{w}\|^2$ instead, as it is a convex function and gives the same result. This leads to the formulation of the hard-margin SVM optimization problem.

11.3 The Primal Optimization Problem (Hard Margin)

$$\begin{aligned} \underset{\mathbf{w},b}{\text{minimize}} \quad & \frac{1}{2}\|\mathbf{w}\|^2 \\ \text{subject to} \quad & y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1, \quad \forall i \end{aligned}$$

This is a convex optimization problem, meaning it has no local minima, only a single global minimum. It can be solved using techniques like Quadratic Programming (QP).

12 Soft-Margin SVM: Handling Non-Separable Data

The hard-margin formulation assumes the data is perfectly linearly separable. In practice, this is often not the case due to noise or overlapping classes. To handle this, we introduce **slack variables**, denoted by $\xi_i \geq 0$.

A slack variable measures how much a data point $\mathbf{x}_i$ violates the margin.

- If $\xi_i = 0$, the point is correctly classified and on or outside the margin.
- If $0 < \xi_i \leq 1$, the point is correctly classified but falls inside the margin.
- If $\xi_i > 1$, the point is misclassified.

The new constraint becomes:

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i$$

We must modify our objective function to penalize these margin violations. This gives us the **soft-margin SVM** formulation.

12.1 The Primal Optimization Problem (Soft Margin)

$$\begin{aligned} \underset{\mathbf{w},b,\xi}{\text{minimize}} \quad & \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N}\xi_i \\ \text{subject to} \quad & y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i, \quad \forall i \\ & \xi_i \geq 0, \quad \forall i \end{aligned}$$

The hyperparameter $C > 0$ controls the trade-off:

- **Large C:** High penalty for margin violations. The model tries to classify every point correctly, potentially leading to a smaller margin and overfitting.
- **Small C:** Low penalty for margin violations. The model prioritizes a larger margin, even if it means misclassifying some points, which can lead to better generalization.

This formulation is equivalent to optimizing for the **hinge loss**, which is defined as $\max(0, 1 - y_i(\mathbf{w}^T\mathbf{x}_i + b))$.

13 The Dual Form and the Kernel Trick

While the primal form is intuitive, solving it can be complex. The **dual form**, derived using Lagrange multipliers, is often more efficient and provides a powerful insight.

13.1 Deriving the Dual

By setting up the Lagrangian for the primal problem and taking derivatives with respect to $\mathbf{w}$ and b, we find that the optimal weight vector $\mathbf{w}$ is a linear combination of the data points:

$$\mathbf{w} = \sum_{i=1}^{N} \alpha_i y_i \mathbf{x}_i$$

where α_i are the Lagrange multipliers. Crucially, the α_i values are non-zero **only for the support vectors**. This means the solution is determined only by a subset of the training data.

Substituting this back into the Lagrangian leads to the dual optimization problem:

$$\begin{aligned} \underset{\alpha}{\text{maximize}} \quad & \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j (\mathbf{x}_i^T \mathbf{x}_j) \\ \text{subject to} \quad & \sum_{i=1}^{N} \alpha_i y_i = 0 \\ & 0 \leq \alpha_i \leq C, \quad \forall i \end{aligned}$$

The key observation here is that the data points $\mathbf{x}_i$ only appear in the form of a dot product $(\mathbf{x}_i^T \mathbf{x}_j)$.

13.2 Non-Linear SVM and the Kernel Trick

What if the data is not linearly separable, even with slack variables? A powerful strategy is to map the data into a higher-dimensional space where it becomes linearly separable. Let $\Phi(\mathbf{x})$ be a function that maps $\mathbf{x}$ to this higher-dimensional space.

The dual formulation in this new space would involve the dot product $\Phi(\mathbf{x}_i)^T \Phi(\mathbf{x}_j)$. Computing this can be very expensive, especially if the new dimension is very high or infinite.

This is where the **kernel trick** comes in. A kernel function, $K(\mathbf{x}_i, \mathbf{x}_j)$, computes the dot product in the higher-dimensional space without ever explicitly performing the transformation:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \Phi(\mathbf{x}_i)^T \Phi(\mathbf{x}_j)$$

By simply replacing $\mathbf{x}_i^T \mathbf{x}_j$ with $K(\mathbf{x}_i, \mathbf{x}_j)$ in the dual formulation, we can create a non-linear SVM.

13.3 Common Kernel Functions

Any function that satisfies Mercer's condition can be used as a kernel. Some popular kernels include:

- **Linear Kernel:** $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$. This is the standard linear SVM.
- **Polynomial Kernel:** $K(\mathbf{x}_i, \mathbf{x}_j) = (\gamma \mathbf{x}_i^T \mathbf{x}_j + r)^d$. This creates polynomial decision boundaries.
- **Radial Basis Function (RBF) Kernel:** $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$. This is a powerful, general-purpose kernel that can create complex decision boundaries. The hyperparameter γ controls the influence of each support vector. This kernel maps to an infinite-dimensional space.

14 One-Class SVM

Standard Support Vector Machines are designed for binary classification, distinguishing between two well-defined classes. However, in many real-world scenarios, we may possess abundant data for one class (the "positive" class) but very little or unreliable data for the other (the "negative" class). This specific problem is addressed by the **One-Class SVM**.

14.1 Motivation

A classic example of this problem is an email spam filter or an anomaly detection system:

- We often have a vast dataset of "good" emails (positive examples).
- Modeling "spam" (negative examples) is difficult because spammers constantly evolve their methods to evade detection, making it hard to acquire a representative set of negative examples.

To solve this, the One-Class SVM trains solely on the positive class. It builds a model of what the "normal" data looks like. Any new data point that deviates significantly from this model is subsequently classified as an outlier or negative example.

14.2 Geometric Interpretation

The One-Class SVM operates by mapping the input data into a high-dimensional feature space using a kernel function, $K(x, x')$.

- The algorithm attempts to separate the training data from the **origin** in this mapped space.
- The goal is to find a hyperplane that maximizes the distance (margin) between the data points and the origin.

14.3 Mathematical Formulation (Nu-SVM)

The objective function for the One-Class SVM (specifically the ν-SVM formulation) involves minimizing the weights and slack variables while maximizing the margin ρ.

$$\text{Minimize} \quad \frac{1}{2}\mathbf{w}^T\mathbf{w} + \frac{1}{\nu n}\sum_{i=1}^{n}\epsilon_i - \rho \tag{15}$$

Subject to the constraints:

$$\mathbf{w}^T\mathbf{x}_i + b \geq \rho - \epsilon_i \tag{16}$$

$$\epsilon_i \geq 0 \quad \forall i \tag{17}$$

Key definitions:

- $\mathbf{w}$: The weight vector perpendicular to the decision boundary.
- ρ (rho): The radius or offset of the decision boundary from the origin that we optimize.
- ϵ_i: The slack variables allowing for some misclassifications (outliers).
- ν (nu): A hyperparameter that sets an **upper bound** on the fraction of training examples that can be regarded as outliers (negative).

15 Summary of SVM

- **Strengths:**
  - Effective in high-dimensional spaces.
  - Memory efficient as it only uses a subset of training points (support vectors).
  - It is a convex optimization problem, guaranteeing a global optimum.
  - Versatile through the use of different kernels.
- **Weaknesses:**
  - Does not scale well to very large datasets, as computing the Gram matrix ($K(\mathbf{x}_i, \mathbf{x}_j)$ for all pairs) can be $O(N^2)$.
  - Choosing the right kernel and hyperparameters (like C and γ) can be challenging.
- **One-Class SVM:** A variant used for unsupervised anomaly detection. It learns a boundary around a single class of data, identifying any points outside this boundary as outliers.

SVMs are a powerful and foundational model in machine learning, and the concept of maximizing margins remains influential in modern deep learning architectures.

References

[1] Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction, 2020.

[2] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9(86):2579–2605, 2008.